

软件构造期末总结

第一章

面向对象方法

三大特性:

- 封装：隐藏对象的属性和实现细节，仅对外公开访问方法;增强安全性和简化编程
- 继承：子类继承父类的特征和行为；实现代码的复用
- 多态：同一个行为具有多个不同表现形态的能力（“一个接口，多个方法”），提高了程序的扩展性和可维护性

代码示例：

```
// 父类：动物
class Animal {
    private String name; // 封装：属性私有化

    public Animal(String name) {
        this.name = name;
    }

    // 封装：通过公开方法访问私有属性
    public String getName() {
        return name;
    }

    // 多态：父类提供通用方法
    public void speak() {
        System.out.println(name + " 发出声音");
    }
}

// 子类：狗，继承 Animal
class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }

    // 方法重写：狗有自己的叫声
    @Override
    public void speak() {
        System.out.println(getName() + " 汪汪叫");
    }
}

// 子类：猫，继承 Animal
class Cat extends Animal {
    public Cat(String name) {
        super(name);
    }

    // 方法重写：猫有自己的叫声
    @Override
    public void speak() {
        System.out.println(getName() + " 喵喵叫");
    }
}
```

```
}

public class Main {
    public static void main(String[] args) {
        // 多态：父类引用指向子类对象
        Animal a1 = new Dog("小黑");
        Animal a2 = new Cat("小白");

        a1.speak(); // 输出：小黑 汪汪叫
        a2.speak(); // 输出：小白 喵喵叫
    }
}
```

面向对象思想

要点:

- 任何事物都是对象，对象有属性和方法。复杂对象可以由相对简单的对象,以某种方式构成。
- 通过类比发现对象间的相似性，即对象间的共同属性，是构成对象类的依据。
- 对象间的相互联系是通过传递“消息”来完成的。通过对象之间的消息通信，驱动对象执行一系列的操作从而完成某一任务。

优点:

- 模块化
- 自然性
- 并发性
- 重用性

第二章

基本数据类型

标识符规定：字母，下划线，\$开头都可以，但不能用数字开头

- 基本数据类型
 - 数值型: 整数类型(byte short int long) 浮点类型(float double)
- 引用数据类型
 - class
 - interface

- 。 数组

类型	空间	范围	备注
byte	1 字节	-128 — 127	
short	2 字节	-2^{15} — $2^{15} - 1$	
int	4 字节	-2^{31} — $2^{31} - 1$	
long	8 字节	-2^{63} — $2^{63} - 1$	声明常量要加「L」
float	4 字节	— —	声明常量要加「F」
double	8 字节	— —	浮点数的默认类型
char	16 字节	UTF-16	
boolean	— —	false 或 true	不能用 0 和非 0 代替

各种标识符要点

float 和 long 构造方法

一定要在数字后面加 **f/F** 或 **L/l**

```
float f = 3.14f
long l = 100l
```

关于引用类型的解释

如果把一个变量的名字比作门牌号，房子本身比作变量的值，引用相当于把门牌号换了，但是你改变的还是房子里的东西。

```
Student s1 = new Student("张三");
Student s2 = s1;
s2.name = "李四";
System.out.println(s1.name); // 输出李四
```

boolean类型的要点

赋值只能用 true 或 false ，不能用数字！

控制流程

与 C语言大差不差

- 条件语句
 - if-else
 - switch
- 循环语句
 - for
 - while
 - do-while
- 跳转语句
 - break
 - continue
 - return

输入与输出

标准输入 **System.in**

1. 使用字符流对System.in进行封装
记住BufferedReader关键字

```
BufferedReader stdin = new BufferedReader(new InputStreamReader(System.in));  
System.out.print("enter a Line: ")  
System.out.println(stdin.readLine());
```

2. 使用java.util.Scanner对System.in进行封装
记住 Scanner 关键字

```
Scanner stdin = new Scanner(System.in);  
System.out.print("enter a Line: ");  
System.out.println(stdin.nextLine()); // 读取一行输入并获取字符串
```

标准输出 **System.out**

- println: 输出换行
- print 不换行
- write 输出字节数组, 也不换行

数组

创建方式

```
int[] a;//java风格
int b[];//C语言风格
int[] c = new int[2]; //一开始制定数组的长度
int[] d = new int[]{0,1}; //直接给数组赋值
```

warning : 数组一经创建, 长度无法更改

异常机制

异常关键字

- try: 用于监听。将要被监听的代码(可能抛出异常的代码)放在try语句块之内, 当try语句块内发生异常时, 异常就被抛出。
- catch: 用于捕获异常。catch用来捕获try语句块中发生的异常。
- finally: finally语句块总是会被执行。它主要用于回收在try块里打开的资源(如数据库连接、网络连接和磁盘文件)。只有finally块执行完成之后, 才会回来执行try或者catch块中的return或者throw语句。如果finally中使用了return或者throw等终止方法的语句, 则就不会跳回执行, 直接停止。
- throws: 用在方法签名中, 用于声明该方法可能抛出的异常。
- throw: 用于抛出异常。

throw与throws

共同点: 都负责抛出异常

不同点: throws 用于方法头, 只是一个申明, throw 用于内部, 抛出异常对象。throws 一次性抛出多个异常, throw 只能一个。

finally

finally 的代码一定会被执行!

JVM与垃圾回收

JVM 特点:

- 一次编译, 到处运行
- 自动内存管理
- 自动垃圾回收

第三章

对象与类

类与对象的比较:

- 类是静态的，类的存在，语义，关系在程序执行前就已经定义好了。
- 对象是动态的，在程序执行的时候可以被创建修改删除。

类的声明和构造

类修饰符

- public : 公共类
- abstract: 抽象类(继承)
- final:最终类(非继承)

访问控制符号

- public
- private
- protected
- default

this关键字

```
public class Person {  
    private String name;  
    private int age;  
    public Person(String name, int age) {  
        this.name = name; // this.name 指的是 class 里的 name, 另一个指的是函数中的参数 name  
        this.age = age;  
    }  
  
    Person Li = new Person("Li Lei",19); // 创建的时候直接传入参数  
}
```

继承:

子类不能直接extends继承父类的构造方法,子类需要使用super语句来继承父类的构造方法

```
class Person {
String name;
int age;
String getname() {

}
public Person(String name, int age) {

}
}

class Student extends Person {
super(name,age);
String school;
String getschool(){

}
}
```

类的访问域

熟记此图即可：

访问范围	private	default	protected	public
同一个类中	√	√	√	√
同一个包中	×	√	√	√
子类中	×	×	√	√
全局范围内	×	×	×	√

static 修饰符

在类中用 static 修饰符声明的属性是共享的

warning:

- 使用static声明的方法，不能访问非static的操作（属性或方法）
- 非static声明的方法，可以访问static声明的属性或方法

原因:

- 如果一个类中的属性和方法都是非static类型的，一定要有实例化对象才可以调用
- Static声明的属性或方法可以通过类名访问，可以在没有实例化对象的情况下调用

静态块

- 一个程序可以有多个 static 块
- 在类被加载的时候执行且仅会被执行一次,按照static块的顺序来执行每个static块

```
class Person
{
    public Person(String name){

    }
    static
    {
        System.out.println("this is a test");
    }
}

Person p1=new Person("aa");
Person p2=new Person("bb" );
// 最终只会输出一次 this is a test
```

数组

warning

- 初始化要写明数组的大小
- 从数组底层的运行机制来看，没有多维数组,本质是一堆连续的内存
- 多维数组中，第二维的长度可以不相等
- `int[ ][ ]arr = new int[ ][3]` 非法，要先声明一维数组的长度

第四章

继承

extends与super

- 继承利用 extends 关键字
- 子类不能直接继承父类的构造方法，需利用super关键字

```
public class Person{
    private String name;
    private int age;

    public Person(String name, int age){
        this.name = name;
        this.age = age;
    }

    public String getName(){
        return name;
    }

    private String setName (String name){
        this.name = name;
    }
}

public class Student extends Person{

    String school;

    public Student(String name, int age,
        String school) {

        super(name, age); // 调用父类的构造方法
        this.school = school;
    }

    public string getSchool(){
        return this.school;
    }
}
```

接口与抽象类

抽象类

抽象类不能实例化

```
//定义一个抽象类
public abstract class Person{

    //普通方法
    public String getName()
    {
        return name;
    }

    //抽象方法
    //没有方法体，用abstract做修饰
    public abstract void getMission();
}
```

```
Person P = new Person(); //错误，抽象类不能实例化
```

每一个抽象方法都是充当占位的功能，功能的具体实现要到子类。

接口

比抽象类还抽象：没有字段，所有方法都是抽象方法

```
interface Person{
    void getDuty();
    void getMission();
}
```

当class要实现一个 interface 的时候，使用 implements 关键字

```
public class Student implements Person{
    private String name;
    private int age;

    @Override
    public void getDuty(){
        System.out.println("好好学习");
    }

    @Override
    public void getMission(){
        System.out.println("为中华之崛起而读书");
    }
}
```

接口 vs 抽象类

特性	Abstract Class	Interface
抽象方法	可以定义抽象方法	可以定义抽象方法
字段	可以定义字段	无字段
继承	只能 extends 一个 class	可以 implements 多个 interface

多态与重写

重写

当父类的某些方法对子类不适用，需要重写(override)父类中的这个方法

```

public class Person{

    private String name;
    private int age;

    public void getTarget(){
        System.out.println("美好生活");
    }
}
// Student 重写 getTarget
public class Student extends Person{
    @Override
    public void getTarget(){
        System.out.println("功夫到家");
    }
}

```

抽象类

```

public class Person{

    private String name;
    private int age;

    public void getTarget(){
        System.out.println("美好生活");
    }
}

public class Engineer extends Person{

    @Override
    public void getTarget(){
        System.out.println("技术改变世界");
    }
}

Person Wang = new Engineer();

```

多态的定义格式: 父类类名 引用名称 = new 子类类名();

多继承问题

内部类:

在类内部不仅可以定义成员变量和方法，还可以定义另一个类。如果在类A的内部再定义一个类B，则B类就称为内部类，而A类则称为外部类。

```
public class Father{

    public int strong(){
        // 强壮指数
        return 9;
    }
}

public class Mother{

    public int smart(){
        // 聪慧指数
        return 8;
    }
}

public class Son {
    // 内部类继承 Father 类
    class Father_Inner extends Father {

        public int strong() {
            return super.strong() + 1;
        }

    }

    // 内部类继承 Mother 类
    class Mother_Inner extends Mother {
        public int smart() {
            return super.smart() + 2;
        }
    }
}
```

接口

```

public interface Father{
    public void strong();
}
public interface Mother{
    public void smart();
}
public interface Daughter extends Father, Mother{
    public void kind();
}
public class Girl implements Daughter{
    public static void main(String[] args){

    }
    @Override
    public void strong(){
        System.out.println("She's not strong.");
    }
    @Override
    public void smart(){
        System.out.println("She's very smart.");
    }
    @Override
    public void kind(){
        System.out.println("She's very kind.");
    }
}

```

超类与 super 关键字

Object超类

Object超类可以显式继承，也可以隐式继承，两种方式均可：即写不写 extends 关键字都没关系

可以用Object类型的变量引用任何的对象： `Object obj = new Student("Zhangsan", 19);`

equals方法

用于比较两个对象是否相等

- 不同对象，内存地址不同，不相等，返回 false
- 对象引用，内存地址相同，相等，返回 true

super 关键字

使用 super 关键字可以访问父类当中的变量和方法(当子类中有与父类相同的变量名或方法名的时候)

```
class Person {  
    int age = 39;  
}  
  
class Student extends Person {  
    int age = 18;  
  
    void display() {  
        System.out.println("学生年龄: " + super.age);  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Student stu = new Student();  
        stu.display();  
    }  
}
```

异常的继承框架

可以像下面这样自定义异常类：

```
class MyException extends Exception{  
}
```

- 如果希望自定义一个检查性异常类，则需要继承Exception类。
- 如果希望自定义一个运行时异常类，那么需要继承RuntimeException类。

第五章

面向对象设计原则

- 单一职责原则：一个类而言，仅有一个引起它变化的原因。即：一个类该干几件事？
- 开闭原则：一个软件实体应当对扩展开放，对修改关闭。即在设计一个模块的时候，应当使这个模块可以在不被修改的前提下被扩展。即：增加新功能时是否需要修改旧代码？
- 里氏代换原则：子类应该能够替换父类，并且不会导致程序出错。

- 依赖倒转原理：面向接口编程，而不是面向具体实现编程。好比在 Computer 这个类当中，应该定义接口 CPU,让接口 CPU 去处理 AMD,Intel 这些分类别，而不是直接在 Computer 类中写
- 合成/聚合复用原则：尽量使用对象组合，而不是继承来达到复用的目的。
- 接口隔离原则：客户端不应该依赖它不需要的接口。
- 迪米特法则：一个软件实体应当尽可能少的与其他实体发生相互作用。

设计模式(上)

单例模式

保证一个类仅有一个实例，并提供一个访问它的全局访问点。

比如飞机大战游戏当中只有一个英雄机。

单例模式的实现

- 饿汉式：就是在类加载的时候立刻会实例化，后续使用就只会出现一份实例。

```
public class Singleton {  
    private static Singleton singleton = new Singleton();  
    // 在类加载的时候就创建，同时private 防止被外部调用  
}
```

- 懒汉式：在类加载的时候没有直接实例化，而是调用指定实例方法的时候再进行实例化，好处是比饿汉式效率高

```
public class Singleton {  
  
    private static Singleton instance;  
    private Singleton() {  
    }  
    public static Singleton getInstance() {  
        if(instance == null){  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

- 同步锁：解决懒汉式单例在多线程环境下创建多个实例的问题，运用 Java 的synchronized 同步锁机制。线程安全但效率低。

```

public class Singleton {
    // synchronized 关键字保证同一时刻只允许一个线程进入被保护的代码区域。
    private static Singleton instance;
    private Singleton(){}
    public static synchronized Singleton getInstance() {
        if(instance == null){
            instance = new Singleton();
        }
        return instance;
    }
}

```

设计模式(下)

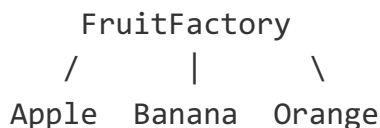
简单工厂模式

例如一个负责计算面积的类，下面直接负责圆，矩形等面积的算，当要新增一个椭圆的计算的时候就需要新增直接修改类，不符合开闭原则。

工厂模式

相比简单工厂，核心思想是不再让一个工厂负责所有产品，而是让每个产品都有自己的工厂。

简单工厂：



工厂方法：

AppleFactory → Apple

BananaFactory → Banana

OrangeFactory → Orange

这样新增产品的时候只会新增产品而不是原工厂。

抽象工厂模式

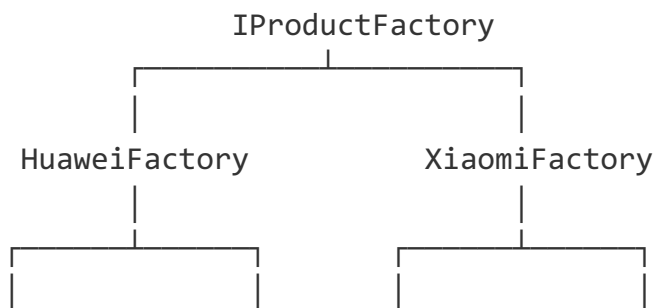
简单工厂：一个工厂生产所有产品

工厂方法：一个工厂生产一种产品

抽象工厂：一个工厂生产一整套关联产品

比如华为和小米同时生产路由器和手机

可以画成这样：



HuaweiPhone HuaweiRouter XiaomiPhone XiaomiRouter

扩展情况：新增一个品牌很容易，例如新增一个 AppleFactory。但是要新增一个产品很难。

第六章

白盒测试

测试人员知道程序内部结构，根据源代码逻辑设计测试用例。

优势：

- 针对性强
- 可以快速定位 Bug
- 有助于了解测试覆盖程度
- 有助于优化代码

劣势：

- 对测试人员要求高
- 成本较高

白盒测试重点：逻辑覆盖

- 语句覆盖
- 判定覆盖
- 条件覆盖
- 判定/条件覆盖
- 条件组合覆盖
- 路径覆盖

它们的覆盖强度是：

语句覆盖 < 判定覆盖 < 条件覆盖 < 判定/条件覆盖 < 条件组合覆盖 < 路径覆盖

其中：

语句覆盖：

至少让每条语句执行一次，覆盖能力最弱。

判定覆盖：

每个判断分支的真、假结果都至少执行一次。

条件覆盖：

每个简单条件的真、假都至少执行一次。

条件组合覆盖：

每个判断中所有条件组合都至少执行一次。

路径覆盖：

每条可能路径都至少执行一次，覆盖能力最强，但实际中通常难以完全实现。

黑盒测试

在完全不考虑程序的内部逻辑结构和内部特性的情况下，测试者在程序接口进行测试，它只检查程序功能是否按照需求规格说明书的规定正常使用，程序是否能适当的接收输入数据而产生正确的输出信息。

优势：

- 方法简单有效
- 可以整体测试系统行为
- 开发与测试可以并行
- 开发与测试可以并行

劣势：

- 覆盖率较低
- 直接依赖于需求规格说明书
- 入门门槛低

第七章

List接口及其标准实现

List接口定义了一个有序的对象集合，允许重复元素存在。

- ArrayList:和普通数组的区别是可以添加删除元素，不受长度限制
- LinkedList：相当于 cpp 的链表，增加和删除的效率更高，查找和修改效率低

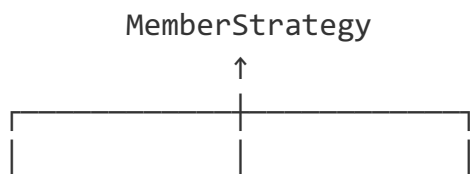
set 和 map 接口

Set集合中的对象不按特定的方式排序，只是简单地把对象加入集合，但Set集合中不能包含重复对象。

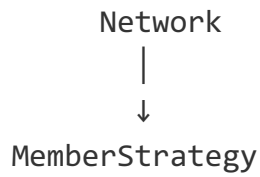
- HashSet:
 - add() 增加元素
 - contains() 判断存在
 - remove() 删除元素
 - clear() 删除所有元素
 - size() 元素数量
- HashMap: key-value 映射，例如 string->number apple->100 banana->101
 - get(key)获取 key 对应的 value
 - remove(key)删除 key 对应值
 - clear() 删除所有元素
 - size() 元素数量

策略模式

可以这样理解：



PrimaryStrategy IntermediateStrategy AdvancedStrategy



对于销售网络，只需要知道有 MemberStrategy 这一个方法，可以打折，具体针对不同会员等级，实现是分开的

优点：

- 提供了一种替代继承的方法，而且既保持了继承的优点（代码重用）还比继承更灵活
- 避免过多的 if-else
- 遵守大部分设计原则，高内聚、低耦合。

缺点：

- 客户端必须知道所有策略类
- 策略类数量会增加

迭代器模式

迭代器可以访问数组，链表，哈希表

相比 for 循环适用范围更广，因为 Set 这一类数据结构没有索引，不适合用 for 访问

例如

```

import java.util.ArrayList;
import java.util.Iterator;

public class IteratorTest {

    public static void main(String[] args) {

        ArrayList<String> sites = new ArrayList<>();

        sites.add("Google");
        sites.add("Amazon");
        sites.add("Taobao");
        sites.add("Zhihu");

        Iterator<String> it = sites.iterator();

        while (it.hasNext()) {
            String site = it.next();
            System.out.println(site);
        }
    }
}

```

优点：

- 隐藏集合内部结构
- 统一遍历接口
- 支持多种遍历方式
- 简化聚合类
- 符合单一职责原则和开闭原则

缺点：

- 简单遍历时较繁琐
- 可能增加类和对象数量
- 对数组、ArrayList 等简单结构未必最高效

第八章

输入输出流

按数据单位可以把流分成：

- 字节流：父类是 `InputStream` `OutputStream` ,以byte为单位，适合处理图片，音频，视频这些文件
- 字符流：父类是 `Reader` `Writer` , 适合处理纯文本

Java 中类名一般也有规律：

以 `Stream` 结尾：字节流

以 `Reader` 结尾：字符输入流

以 `Writer` 结尾：字符输出流

例如：

- `FileInputStream` 文件字节输入流
- `FileOutputStream` 文件字节输出流
- `FileReader` 文件字符输入流
- `FileWriter` 文件字符输出流
- `BufferedReader` 带缓冲的字符输入流
- `InputStreamReader` 字节流转字符流
- `OutputStreamWriter` 字符流写入字节流

系统流

- `System.in`:标准输入流，默认设备是键盘。本身是字节流，如果要按字符或字符串读取，通常要包装成字符流。
- `System.out`: 标准输出流，默认输出到控制台。
- `System.err`:标准错误流，默认设备是控制台

文件的输入输出

- `FileInputStream`: `InputStream f = new FileInputStream("C:/java/hello");` 文件名创建对象，或者
`File f = new File("C:/java/hello"); InputStream in = new FileInputStream(f);` 使用 `File` 创建一个文件对象
- `FileOutputStream`: `OutputStream f = new FileOutputStream("C:/java/hello");` 文件名创建对象，或者
`File f = new File("C:/java/hello"); OutputStream fOut = new FileOutputStream(f);` 使用 `File` 创建一个文件对象

Java流继承框架

Java所有的流类位于java.io包中，都分别继承自以下四种抽象流类型(四大家族)

	字节流	字符流
输入流	InputStream	Reader
输出流	OutputStream	Writer

- 在java中只要“类名”以Stream结尾的都是字节流，以“Reader/Writer”结尾的都是字符流
- 所有的流都实现了java.io.Closeable接口
 - 都是可关闭的，都有close()方法
 - 流毕竟是一个管道，用完之后要关闭，不然会耗费很多资源
- 字节流与字符流是可转换的

操作文件

读写文件

读:

- 读取文件所有内容: `byte[] bytes = Files.readAllBytes(path);`
 - 文本文件中读取内容: `String content = Files.readString(path, charset);`
 - 行序列读取内容: `List<String> lines = Files.readAllLines(path, charset);`
- 写:
- 写出一个字符串到文件中: `Files.writeString(path, content, charset);`
 - 向指定文件追加内容: `Files.write(path, content.getBytes(charset), StandardOpenOption.APPEND);`
 - 将一个行的集合写出到文件中: `Files.write(path, lines, charset);`

复制移动和删除:

- 复制: `Files.copy(fromPath, toPath);`
- 移动: `Files.move(fromPath, toPath);`
- 如果目标路径已经存在, 可使用 `REPLACE_EXISTING` 选项
- 删除: `Files.delete(path);`

对象输入输出流与序列化

- `Java.io.ObjectOutputStream`代表对象输出流, 它的`writeObject(Object obj)` 方法可对参数指定的obj对象进行序列化, 把得到的字节序列写到一个目标输出流中。
- `ObjectOutputStream(OutputStream out)` 创建一个`ObjectOutputStream`使得你可以将对象写出到指定的`OutputStream`。

- `Void writeObject(Object obj)` 写出指定的对象到`ObjectOutputStream`，这个方法将存储指定对象的类、类的签名以及这个类及其超类中所有非静态和非瞬时的域的值。
- `Java.io.ObjectInputStream`代表对象输入流，它的`readObject()`方法可从一个源输入流中读取字节序列，再把它们反序列化为一个对象，并将其返回。
- `ObjectInputStream(InputStream in)` 创建一个`ObjectInputStream`用于从指定的`InputStream`中读回对象信息。
- `Object readObject()` 从`ObjectInputStream`中读入一个对象。特别是，这个方法会读回对象的类、类的签名以及这个类及其超类中所有非静态和非瞬时的域的值。它指定的反序列化允许恢复多个对象引用。